

AI WEATHERWISE

DELIVER INTELLIGENT WEATHER FORECASTS AND INSIGHTS

Team ID: SWTID-2026-4628
Team Leader: Deepak S — deepak0607007@gmail.com
Team Members: Dinesh Kumar. S — sdineshk064@gmail.com
Dhananchezhian. M — dhana2006ms@gmail.com
Franklin Arwonish J — arownisha@gmail.com

TABLE OF CONTENTS

1. Project Overview
2. Scenario-Based Case Study
3. Objectives
4. System Requirements
5. Project Architecture
6. Entity-Relationship Design
7. Roles and Responsibilities
8. User Flow
9. Project Setup and Configuration
10. Backend Development
11. Database Configuration
12. API Documentation
13. API Testing
14. Security and Reliability
15. Project Execution
16. Future Scope
17. Conclusion

1. PROJECT OVERVIEW

AI WeatherWise is a robust RESTful backend platform developed using Node.js and Express.js, with MongoDB and Mongoose ODM for data persistence.

The system provides real-time weather metrics and integrates Google Gemini AI to generate intelligent weather summaries and personalized recommendations based on environmental conditions.

The platform provides:

- Real-time weather information
- AI-generated weather summaries
- Personalized weather recommendations
- User authentication
- Favorite-location management
- JWT-based security
- Password hashing using bcrypt
- Centralized error handling
- Request sanitization
- Fallback functionality when external API keys are unavailable

2. SCENARIO-BASED CASE STUDY

2.1 Background

Alex is a travel enthusiast who frequently visits different cities. Alex needs a simple system to track local weather conditions and receive AI-based activity suggestions according to current weather conditions.

2.2 Problem Statement

The project addresses the following problems:

- Difficulty managing multiple favorite weather locations.
- Lack of personalized recommendations for outdoor activities and clothing.
- Weather information is often presented as raw numerical data.
- Users need a secure system to store and synchronize their preferences.

2.3 Proposed Solution

AI WeatherWise provides a centralized backend service that retrieves weather information and processes it through an AI service.

The system manages users and their favorite locations while generating customized weather summaries and recommendations using Google Gemini AI.

3. OBJECTIVES

The main objectives of AI WeatherWise are:

1. To develop a RESTful backend using Node.js and Express.js.
2. To provide real-time weather information through APIs.
3. To allow users to manage favorite cities.
4. To securely authenticate registered users.
5. To store user and location information in MongoDB.
6. To generate AI-powered weather summaries.
7. To provide personalized weather recommendations.
8. To protect the system using JWT authentication and bcrypt password hashing.

9. To provide centralized error handling and request sanitization.
10. To provide fallback functionality when external API services are unavailable.

4. SYSTEM REQUIREMENTS

4.1 Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Node.js:** Version 16 or above
- **npm:** Version 8 or above
- **MongoDB:** NoSQL database
- **Mongoose:** MongoDB ODM
- **Gemini AI SDK:** Google AI integration
- **Postman:** API testing and verification
- **Visual Studio Code:** Development environment

4.2 Hardware Requirements

- **Processor:** Intel Core i5 8th Generation or above / AMD Ryzen 5 or equivalent
- **RAM:** Minimum 8 GB (Recommended: 16 GB)
- **Storage:** Minimum 1 GB available workspace

5. PROJECT ARCHITECTURE

AI WeatherWise follows an MVC-based architecture that separates application logic, data management and API routing.

5.1 Main Components

Interface Layer: API clients such as Postman or other API testing tools send HTTP requests to the backend and receive JSON responses.

Express Server Gateway: Receives HTTP requests, maps GET, POST, PUT and DELETE methods, parses JSON data, handles CORS, and routes requests to controllers.

Authentication Middleware: Extracts JWT tokens, validates authentication information, and checks authorization for protected routes.

Controller Layer: Controllers process incoming requests and coordinate application logic.

Database Interface: Mongoose communicates between the Node.js application and MongoDB.

AI Service: The AI service uses Google Gemini to generate weather summaries and personalized recommendations based on weather conditions.

6. ENTITY-RELATIONSHIP DESIGN

6.1 User Entity

Field	Type	Description
_id	ObjectId	Primary Key
name	String	Required
email	String	Required and Unique
password	String	Hashed password
role	String	Default: reader

6.2 Location Entity

Field	Type	Description
_id	ObjectId	Primary Key
user	ObjectId	Reference to User
city	String	Required
country	String	Required
createdAt	Date	Default: Current Date

6.3 Relationship

A registered User can have multiple favorite Locations, creating a one-to-many relationship.

7. ROLES AND RESPONSIBILITIES

7.1 Admin

The administrator is responsible for:

- Managing user accounts.
- Monitoring API health.
- Monitoring system logs.
- Maintaining database integrity.
- Managing system-level operations.

7.2 Registered User

A registered user can:

- Create and manage favorite locations.
- Request real-time weather information.

- Request AI weather recommendations.
- Receive personalized weather summaries.
- Manage authentication credentials.

7.3 Public Weather User

A public user can access public weather endpoints and consume current weather metrics without necessarily having an account.

8. USER FLOW

The general system workflow is:

1. User registers or logs in.
2. The authentication system validates the user.
3. The user saves a favorite city.
4. The user requests current weather information.
5. The backend retrieves the weather data.
6. The user requests AI-based weather insights.
7. The system sends relevant weather information to the AI service.
8. Gemini generates a summary or recommendation.
9. The backend returns the result as a JSON response.

9. PROJECT SETUP AND CONFIGURATION

9.1 Creating the Project

Create a project folder named **AI WeatherWise** and open it using Visual Studio Code.

Initialize the Node.js project:

```
npm init -y
```

9.2 Installing Dependencies

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

Install Nodemon for development:

```
npm install --save-dev nodemon
```

9.3 Project Files

The project contains important files such as:

- index.js
- .env
- package.json
- .gitignore

9.4 Environment Configuration

The `.env` file contains environment-specific configuration:

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

API keys and secret values should be kept private and should not be uploaded to public repositories.

10. BACKEND DEVELOPMENT

The backend is developed using Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini AI.

The backend follows a modular structure separating:

- Routes
- Controllers
- Middleware
- Models
- Services
- Utility modules

This modular architecture improves maintainability and scalability.

10.1 Application Entry Point

The `index.js` file acts as the main entry point. It performs the following tasks:

1. Loads environment variables.
2. Initializes Express.
3. Configures middleware.
4. Connects to MongoDB.
5. Registers API routes.
6. Starts the backend server.

11. DATABASE CONFIGURATION

MongoDB is used as the database for the project. Mongoose provides communication between the Node.js application and MongoDB.

11.1 Database Connection

The database connection should be established before processing backend operations.

11.2 Mongoose Schemas

Two important entities are **User** and **Location**. The User schema stores authentication and profile information, while the Location schema stores the cities saved by users. The database design maintains the relationship between users and their favorite locations.

12. API DOCUMENTATION

12.1 Authentication API - Register

POST /api/auth/register

Purpose: Creates a new user account.

Request Body:

```
{
  "name": "User Name",
  "email": "user@example.com",
  "password": "password"
}
```

Function: Validates user information, hashes the password, creates a user record, and returns the registration result.

12.2 Login API

POST /api/auth/login

Purpose: Authenticates an existing user.

Request Body:

```
{
  "email": "user@example.com",
  "password": "password"
}
```

Function: Checks user credentials, compares password using bcrypt, generates a JWT token after successful authentication, and returns the authentication response.

12.3 Weather API

GET /api/weather/Chennai

Purpose: Retrieves current weather information for a city.

Typical Response Structure:

```
{
  "city": "Chennai",
  "temperature": 32.99,
  "humidity": 67,
  "windSpeed": 3.81,
  "condition": "Cloud",
  "isMock": false
}
```

The response provides structured weather information such as city, temperature, humidity, wind speed and weather condition.

12.4 AI Weather Recommendation API

POST /api/recommendation

Purpose: Generates an AI-powered recommendation based on weather information.

Example Request:

```
{
  "temperature": 32.99,
  "humidity": 67,
  "windSpeed": 3.81,
  "condition": "Cloud"
}
```

Response:

```
{
  "recommendation": "AI-generated weather recommendation"
}
```

13. API TESTING

The APIs can be tested using Postman or Thunder Client. The testing process includes:

13.1 Register API Testing

The registration endpoint is tested by sending user details through a POST request. The system validates the information and creates the user account.

13.2 Login API Testing

The login endpoint is tested using registered user credentials. A successful login generates an authentication token.

13.3 Weather API Testing

The Weather API is tested by providing a city name. The backend returns weather information in JSON format (City, Temperature, Humidity, Wind speed, Weather condition, Mock-data status).

13.4 Recommendation API Testing

The recommendation endpoint receives weather information and generates an AI-based recommendation returned in JSON format.

13.5 Error Testing

Invalid requests are handled by the backend through centralized error handling, request sanitization, and appropriate HTTP responses.

14. SECURITY AND RELIABILITY

AI WeatherWise includes several security and reliability mechanisms:

- **14.1 JWT Authentication:** Used for authentication and protection of secured routes.
- **14.2 Password Hashing:** Passwords are protected using bcrypt hashing instead of plain-text.

- **14.3 Role-Based Route Protection:** Middleware evaluates authentication and authorization before resource access.
- **14.4 Request Sanitization:** Prevents XSS attacks, NoSQL parameter manipulation, and unsafe input processing.
- **14.5 Error Handling:** Centralized error handling provides consistent error responses.
- **14.6 Fallback Mode:** Maintains API functionality when external API keys or services are unavailable.

15. PROJECT EXECUTION

Before running the project:

1. Install all dependencies.
2. Start MongoDB.
3. Configure the `.env` file.
4. Verify the database connection.
5. Start the Node.js backend.

Install dependencies and start development server:

```
npm install
npm start
```

After successful execution, the backend connects to MongoDB and begins listening for API requests.

16. FUTURE SCOPE

The project can be extended with the following features:

- Web-based frontend and Mobile application.
- Multi-day weather forecasts and Location-based weather detection.
- Weather alerts, severe-weather warnings, and notifications.
- More personalized AI recommendations based on weather history & analytics.
- Multiple saved locations and dashboards.
- Automated API testing, cloud deployment, production monitoring and logging.

17. CONCLUSION

AI WeatherWise is an intelligent weather backend designed to combine real-time weather information with AI-powered insights.

The system uses Node.js and Express.js for backend development, MongoDB and Mongoose for data persistence, JWT and bcrypt for authentication and security, and Google Gemini AI for intelligent weather summaries and personalized recommendations.

The modular MVC-based architecture makes the system easier to maintain and extend. The RESTful API structure also allows the backend to be integrated with web applications, mobile applications and other client platforms.

Overall, AI WeatherWise provides a foundation for delivering weather information in a more understandable and personalized form rather than presenting users with raw weather metrics alone.